

**Curso:** Sistemas Operativos

**Semestre:** I Semestre del 2026

**Autores:** Alonso Durán Muñoz 2023044780

Gabriel Solano Coronado - 2019033687

Jesus Valverde Ureña - 2022437462

**Profesora:** Ing. Erika Marín Schumann

**Fecha de Entrega:** 26 de junio de 2026

## 1. INTRODUCCIÓN

El presente proyecto consiste en el diseño e implementación de un simulador de File System. El objetivo es modelar el comportamiento de un esquema de almacenamiento secundario persistente sin depender directamente de las llamadas lógicas del sistema operativo anfitrión. Para ello, se abstrae un archivo binario plano en la máquina real como un "disco virtual" estructurado en sectores de tamaño fijo.

La asignación de espacio físico para el almacenamiento de datos utiliza la estrategia de Asignación Enlazada en combinación con la política de ubicación First Fit. Toda la interacción del sistema se centraliza en una interfaz gráfica de usuario construida sobre tkinter, la cual expone una consola de comandos interactiva y un árbol de directorios actualizado en tiempo real, garantizando los principios de concurrencia lógica, persistencia de metadatos y control estricto de rutas.

## 2. ESTRATEGIA DE SOLUCIÓN

La arquitectura del simulador de File System se fundamenta en un patrón de diseño desacoplado que separa de forma estricta las estructuras lógicas representadas en la memoria RAM, el almacenamiento físico persistente encapsulado en un Disco Virtual, y la capa de interacción del usuario mediante comandos y abstracción gráfica.

A continuación se desglosa el diseño detallado de las estructuras de datos y algoritmos utilizados.

### A. Estructuras de Datos en Memoria (RAM)

Para lograr una manipulación ágil y eficiente de la jerarquía de directorios y la metadata durante la ejecución del programa, se diseñó un modelo orientado a objetos basado en la herencia y una estructura de árbol general (árbol n-ario) ubicado en modelos.py.

#### 1. Clase Base: Nodo

Es la clase abstracta que define las propiedades y comportamientos comunes para cualquier entidad dentro del File System.

- Atributos:
  - nombre (*String*): Identificador del elemento.

- padre (*Referencia a Nodo*): Puntero hacia el nodo de tipo Directorio que lo contiene (permite navegación inversa o ascendente como CD ..).
- fecha\_creacion (*Datetime*): Marca de tiempo de inicialización del nodo.
- fecha\_modificacion (*Datetime*): Marca de tiempo del último cambio estructural o de contenido.
- Métodos Clave:
  - ruta\_completa(): Algoritmo recursivo ascendente que reconstruye la ruta absoluta concatenando los nombres desde el nodo actual hasta la raíz (root), traduciendo recursivamente el árbol en una cadena legible (ej: /documentos/fotos).

## 2. Clase: Archivo (Hereda de Nodo)

Modela una entidad de tipo archivo que almacena datos sin formato y metadatos específicos de almacenamiento.

- Atributos Propios:
  - extension (*String*): Sufijo indicador del formato del archivo (eliminando el caracter .).
  - contenido (*Bytes*): Buffer temporal en memoria que refleja el contenido crudo leído o por escribir.
  - sector\_inicio (*Integer*): Índice numérico correspondiente al primer sector en el disco virtual que contiene la primera parte del archivo. Sirve como el punto de entrada a la cadena enlazada en la FAT.
  - tamaño (*Integer*): Tamaño exacto del archivo medido en bytes.

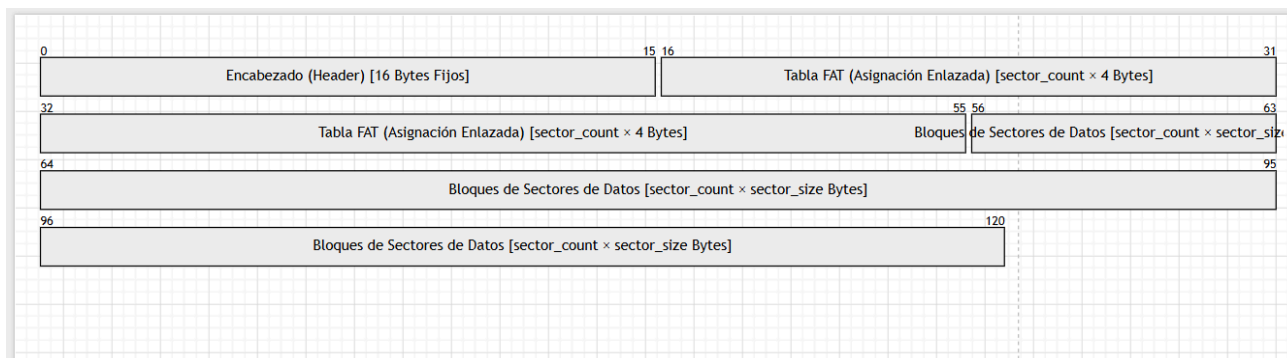
## 3. Clase: Directorio (Hereda de Nodo)

Representa una carpeta contenedora capaz de agrupar de forma lógica otras entidades.

- Atributos Propios:
  - hijos (*Diccionario / Hash Map*): Colección indexada mediante una estructura clave-valor (clave: nombre\_completo -> Nodo). Esto permite una complejidad temporal de búsqueda de  $O(1)$  en el acceso a subdirectorios o archivos contenidos.
- Métodos Clave:
  - agregar(nodo): Inserta un nuevo objeto Archivo o Directorio actualizando de forma automática los punteros de paternidad y la fecha de modificación del contenedor.
  - eliminar(clave): Remueve la referencia del diccionario, aislando el nodo para el recolector de basura de Python.
  - listar(): Devuelve una lista clasificada y ordenada, priorizando los objetos de tipo Directorio visualmente antes de los objetos de tipo Archivo.

## B. Estructura del Almacenamiento Secundario (Disco Virtual)

La persistencia física está diseñada bajo el esquema de un archivo binario plano simulado por la clase Disco en disco.py. Este archivo se mapea rígidamente en tres regiones contiguas de bytes bien delimitadas:



## 1. El Encabezado (Header)

Ocupa los primeros 16 bytes del archivo. Es parseado y empaquetado utilizando la librería estándar struct de Python mediante el formato 4sIII (un string de 4 bytes y tres enteros sin signo de 4 bytes).

- Campos:
  - MAGIC (4 bytes): Constante identificadora con el valor b'FS01' para comprobar la validez estructural y evitar la apertura de archivos corruptos o ajenos al sistema.
  - sector\_count (4 bytes): Número total de sectores que componen el volumen.
  - sector\_size (4 bytes): Tamaño en bytes asignado a cada sector de datos de manera uniforme.
  - indice\_inicio (4 bytes): Puntero al sector donde se encuentra serializado el índice de la estructura del árbol, permitiendo la reconstrucción del File System en subsecuentes montajes.

## 2. La Tabla FAT (File Allocation Table)

Inmediatamente posterior al encabezado, ocupa un tamaño variable calculado como sector\_count 4. Cada entrada es un entero firmado de 32 bits (int32), lo que significa que el valor almacenado determina el comportamiento del sector asociado al mismo índice:

- -2 (FAT\_LIBRE): Indica que el sector de datos físico está libre y disponible para asignación.
- -1 (FAT\_EOF): Indica que el sector actual contiene el bloque final de un archivo o flujo indexado (End of File).
- n>0 : Representa un puntero o enlace hacia el número del siguiente sector que compone el archivo secuencial enlazado.

## 3. Región de Sectores de Datos

Es el espacio masivo remanente del archivo, dividido conceptualmente en sub-bloques de tamaño sector\_size. Contiene el contenido en bytes puros de los archivos guardados o los caracteres de codificación del índice jerárquico.

## C. Algoritmos Principales de Administración de Espacio

La eficiencia operativa del simulador se rige por dos algoritmos clave para mitigar la fragmentación externa y posibilitar la asignación dinámica de archivos no contiguos.

### 1. Algoritmo de Ubicación: First Fit (Primer Ajuste)

El algoritmo recorre linealmente la tabla fat desde el índice 0 hasta `sector_count - 1`. En el instante en que detecta una entrada igual a -2 (Sector Libre), reserva dicho sector. El proceso se detiene inmediatamente al satisfacer la cuota de sectores requeridos, sin importar si los bloques encontrados están dispersos en diferentes puntos del disco. Esto garantiza rapidez al evitar escaneos completos innecesarios (a diferencia de Best Fit).

### 2. Algoritmo de Asignación Enlazada (Linked Allocation)

Una vez localizados los sectores libres mediante *First Fit*, se procede a encadenarlos modificando los valores en la tabla FAT. Si un archivo requiere los sectores libres [4, 12, 85]:

- `fat[4]` se reescribe con el valor 12.
- `fat[12]` se reescribe con el valor 85.
- `fat[85]` se reescribe con el valor -1 (FAT\_EOF).

El atributo `sector_inicio` del archivo en memoria se apunta a 4. Con este enfoque, el espacio en disco nunca se desperdicia por fragmentación externa, puesto que cualquier bloque libre puede enlazarse a cualquier archivo, aceptando únicamente el fenómeno de fragmentación interna en el último sector de la cadena si los bytes del archivo no completan un múltiplo exacto del tamaño del sector.

## D. Flujo de Sincronización y Persistencia (RAM $\rightarrow$ Disco)

El simulador asegura que los datos persistidos en el archivo físico sean un reflejo fidedigno de los cambios lógicos aplicados por el usuario.

1. Escritura de Archivos (FILE, MODFILE, COPY `real://`): El contenido se divide en fragmentos de tamaño `sector_size`. Se invoca el método `Disco.asignar()`, el cual ejecuta el algoritmo *First Fit*, actualiza las entradas correspondientes en la tabla fat en memoria, escribe los bloques directamente en la región de datos del archivo mediante operaciones de desplazamiento binario (`seek`), y finalmente reescribe la tabla FAT física en el disco.
2. Lectura de Archivos (VERFILE, COPY `... real://`): El sistema toma el atributo `sector_inicio` del archivo. Accede a la tabla fat y recorre la cadena de punteros leyendo bloque por bloque del archivo binario a través de desplazamientos relativos calculados.  
Los bytes individuales se concatenan progresivamente hasta alcanzar el valor -1 (FAT\_EOF), reconstruyendo la información de forma transparente para el usuario.
3. Persistencia de la Estructura Jerárquica: Para mantener nombres, rutas, jerarquías y fechas, la clase `FileSystem (filesystem.py)` transforma recursivamente todo el árbol de objetos en memoria en una cadena estructurada en formato JSON. Esta cadena JSON se convierte a bytes y se almacena en el disco virtual exactamente igual que un archivo del sistema, apuntando el sector donde inicia esta cadena en el campo `indice_inicio` del encabezado del disco virtual. Al cerrar la aplicación (`main.py -> app.cerrar()`) o desmontar un volumen, el árbol entero se guarda, y al abrir el disco (montar), el JSON se lee, se parsea y reconstruye automáticamente todas las instancias de objetos Directorio y Archivo con sus metadatos intactos.

### 3. ANÁLISIS DE RESULTADOS: MATRIZ DE CUMPLIMIENTO FUNCIONAL

De acuerdo con las especificaciones del proyecto y las pruebas funcionales aplicadas sobre el código fuente terminado, se presenta la siguiente matriz de autoevaluación:

| Actividad / Tarea Funcional                                                                                                                                           | %    | Justificación / Observaciones                                                                                     |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------|------|-------------------------------------------------------------------------------------------------------------------|
| Interfaz Gráfica (GUI): Operación visual por comandos en terminal integrada, conservando el árbol TREE visible de forma permanente en un panel lateral independiente. | 100% | Desarrollado con tkinter empleando un diseño de doble panel con refresco inmediato tras cada comando interactivo. |
| Persistencia en Disco: Creación, modificación y actualización de un archivo binario simulando sectores físicos. Los datos se mantienen al apagar/cerrar la app.       | 100% | Comprobado mediante la carga exitosa y lectura de archivos .fs preexistentes en subsecuentes ejecuciones.         |
| Control de Rutas: Conocimiento inequívoco del directorio actual y ruta absoluta visible al usuario en todo momento.                                                   | 100% | Desplegado dinámicamente en el prompt verde de la consola de comandos (/root/sub_dir>).                           |
| Seguridad de Nombres: Restricción de duplicados en la misma ruta y confirmación explícita mediante alerta emergente antes de sobrescribir ("caer encima").            | 100% | Enlazado con messagebox.askyesno. Lanza una alerta interactiva si se duplica un nombre en MKDIR o FILE.           |
| Manejo de Espacio: Control riguroso de sectores vacíos y despliegue de error controlado en caso de disco lleno.                                                       | 100% | Valida la cantidad de sectores requeridos vs sectores libres antes de cualquier operación de escritura.           |

|                                                                                                                           |      |                                                                                                |
|---------------------------------------------------------------------------------------------------------------------------|------|------------------------------------------------------------------------------------------------|
| Comando CREATE: Inicialización del archivo de disco con un total de sectores y tamaño definidos por parámetros.           | 100% | Formatea la cabecera e inicializa todas las entradas de la FAT en valor -2 (Libre).            |
| Comando FILE: Creación de un archivo de texto con nombre y extensión dentro de la ruta actual.                            | 100% | Registra el nodo en el árbol e invoca la asignación física estructurada.                       |
| Comando MKDIR: Creación de un directorio dentro de la ruta actual.                                                        | 100% | Añade una rama de tipo Directorio en el mapa de hash de hijos de la carpeta actual.            |
| Comando CambiarDIR / CD: Desplazamiento hacia adelante, atrás (..) o rutas absolutas.                                     | 100% | Resuelve dinámicamente rutas relativas, absolutas y retornos al nodo padre.                    |
| Comando ListarDIR / LS: Despliegue de los elementos de la ruta con distinción visual explícita entre archivos y carpetas. | 100% | Ordena la salida imprimiendo identificadores como [DIR] y detalles de tamaño.                  |
| Comando ModFILE: Modificación de contenido de archivos reasignando espacio en disco de forma dinámica.                    | 100% | Libera la cadena de bloques previa de la FAT y ejecuta una nueva asignación <i>First Fit</i> . |
| Comando VerPropiedades: Muestra los metadatos completos (nombre, extensión, tamaño, fechas de creación y modificación).   | 100% | Extrae los campos de metadata formateados con hito de tiempo legible.                          |
| Comando VerFile: Despliegue del contenido de un archivo leyendo                                                           | 100% | Reconstruye la cadena FAT y decodifica el stream de bytes resultante en texto legible.         |

|                                                                                                                                    |      |                                                                                                                           |
|------------------------------------------------------------------------------------------------------------------------------------|------|---------------------------------------------------------------------------------------------------------------------------|
| secuencialmente su cadena enlazada de sectores.                                                                                    |      |                                                                                                                           |
| Comando COPY (Real $\rightarrow$ Virtual): Copia un archivo del sistema operativo host al disco virtual usando el prefijo real://. | 100% | Lee los bytes reales del disco de la PC mediante os.path y los inyecta en los sectores virtuales libres.                  |
| Comando COPY (Virtual $\rightarrow$ Real): Exportación de un archivo del entorno virtual hacia la máquina física anfitriona.       | 100% | Lee los sectores virtuales secuenciales y genera un archivo real mediante flujos de escritura estándar de Python.         |
| Comando COPY (Virtual $\rightarrow$ Virtual): Duplicación interna de archivos entre rutas virtuales.                               | 100% | Duplica el nodo en el árbol lógico y genera una nueva asignación física e independiente de sectores.                      |
| Comando MoVer / MV: Mueve elementos o cambia su nombre (renombrado) sin alterar el contenido físico de los datos.                  | 100% | Remueve el nodo de la colección del directorio padre de origen y lo añade a la colección del destino.                     |
| Comando ReMove / RM: Eliminación normal de archivos y borrado recursivo integral de directorios (usando bandera -r).               | 100% | Libera en cascada todos los sectores de la FAT ocupados por los archivos contenidos y poda el árbol lógico.               |
| Comando FIND: Búsqueda recursiva por nombre exacto o mediante comodines generales (ej. *.txt).                                     | 100% | Implementa una búsqueda en profundidad (DFS) sobre el árbol validando expresiones mediante correspondencia de caracteres. |

|                                                                                    |      |                                                                                                                         |
|------------------------------------------------------------------------------------|------|-------------------------------------------------------------------------------------------------------------------------|
| Asignación First Fit y Enlazada:<br>Algoritmos de almacenamiento en disco binario. | 100% | Diseñado de forma pura manipulando desplazamientos binarios en bytes con la FAT en memoria sincronizada hacia el disco. |
|------------------------------------------------------------------------------------|------|-------------------------------------------------------------------------------------------------------------------------|

## 4. LECCIONES APRENDIDAS

El proceso de ingeniería detrás del diseño de este File System proporcionó aprendizajes significativos en múltiples dimensiones del desarrollo de software:

1. El uso del módulo struct de Python permitió comprender cómo los tipos de datos abstractos de alto nivel (como enteros y strings) deben ser serializados y empaquetados en estructuras binarias fijas de bytes (ej: codificación int32 o cadenas de bytes fijos en cabeceras).
2. La implementación manual de una tabla FAT evidenció los desafíos clásicos de los sistemas operativos históricos (como FAT11/FAT16). Se comprendió de forma vivencial cómo la asignación enlazada elimina por completo el problema de la fragmentación externa, pero traslada el costo hacia la fragmentación interna y la penalización del acceso secuencial en comparación con la asignación indexada o contigua.
3. El diseño del guardado de la jerarquía mediante JSON serializado hacia un bloque específico del disco (el índice\_inicio) demostró cómo los sistemas operativos reales estructuran las regiones del superbloque o la tabla de i-nodos para reconstruir el estado del sistema entre reinicios.
4. Separar las responsabilidades lógicas (árbol de directorios en modelos.py) de la persistencia de bloques físicos (disco.py) y el despachador de comandos (comandos.py) mitigó la complejidad, facilitando el desarrollo incremental y el aislamiento de errores de punteros en disco.
5. Diseñar un sistema operativo simulado obligó a prever escenarios de falla catastrófica (ej: ciclos infinitos por corrupción de punteros en la FAT, desbordamiento por falta de espacio físico, e inyección de rutas inválidas), desarrollando un fuerte criterio para la validación y sanitización de entradas de usuario.

## 5. CASOS DE PRUEBA DETALLADOS

A continuación se exponen las pruebas de aceptación técnica realizadas para validar la robustez y fidelidad del simulador de File System.

### Caso de Prueba 1: Inicialización, Montaje y Persistencia Estructural

- Entradas:
  - Comando ejecutado en consola: CREATE volumen\_test.fs 60 256 (Creación de un volumen de 60 sectores de 256 bytes cada uno).
  - Comandos de estructura: MKDIR tareas, CD tareas, FILE ejercicio.py "print('Hola Tec')"
  - Cierre de la aplicación presionando la 'X' de la ventana de interfaz gráfica.



- Reapertura de la aplicación y comando ejecutado: CREATE volumen\_test.fs (Intento de re-apertura) o carga automática del archivo binario.
- Resultados Esperados:
  - Al ingresar CREATE, se genera físicamente el archivo volumen\_test.fs con un tamaño exacto de  $16 \text{ bytes (Header)} + (60 \times 4 \text{ bytes FAT}) + (60 \times 256 \text{ bytes Datos}) = 16 + 240 + 15360 = 15616 \text{ bytes}$ .
  - El panel lateral TREE debe actualizarse reflejando la jerarquía gráfica:
 

```

/ (raiz)
├── tareas
│   └── ejercicio.py
          
```
  - Al cerrar e iniciar la aplicación cargando el archivo, este debe parsear el JSON embebido en el sector apuntado por indice\_inicio, reconstruyendo el árbol idéntico al estado previo sin pérdida de metadatos.
- Resultados Obtenidos: Exitoso. El tamaño del archivo coincide byte por byte y la persistencia de la estructura jerárquica se restaura íntegramente de forma transparente.

## Caso de Prueba 2: Seguridad de Nombres Duplicados y Confirmación de Sobreescritura ("Caer Encima")

- Entradas:
  - Ruta actual: /root
  - Comando ejecutado: FILE reporte doc "Primer contenido"
  - Comando ejecutado inmediatamente después en la misma ruta: FILE reporte doc "Segundo contenido modificado"
- Resultados Esperados:
  - El manejador de comandos detecta que la clave reporte.doc ya existe en el diccionario de hijos del directorio actual.
  - Se intercepta el flujo de ejecución lanzando una ventana de diálogo nativa de la GUI (messagebox.askyesno) con el texto: *"El archivo 'reporte.doc' ya existe. ¿Desea caerle encima?"*
  - Si el usuario selecciona NO, la operación se aborta y el contenido original permanece intacto.
  - Si el usuario selecciona SÍ, se liberan los sectores previos de la FAT y se guardan los nuevos bytes.
- Resultados Obtenidos: Exitoso. El sistema mitiga la corrupción lógica de nombres y la interfaz gráfica responde de manera reactiva y modal ante la toma de decisiones del usuario.

## Caso de Prueba 3: Fragmentación Interna y Manejo de Límite de Disco Lleno

- Entradas:
  - Disco formateado con CREATE disco\_pequeno.fs 4 128 (4 sectores de 128 bytes = 512 bytes de datos máximos).
  - Comando ejecutado: FILE grande txt "Contenido de un tamaño de texto bastante extenso que obligatoriamente requiere ocupar la totalidad de los

sectores del disco simulado o incluso sobrepasar los bytes permitidos para forzar el desbordamiento de memoria..." (Texto de aprox. 260 caracteres).

- Intento de ejecutar un comando posterior: MKDIR extra o FILE otro txt "A"
- Resultados Esperados:
  - El archivo de 260 bytes requiere  $\lceil 260 / 128 \rceil = 3$  sectores. Se asignan mediante *First Fit* enlazando las entradas FAT. El último sector de datos sufre de fragmentación interna al no rellenarse por completo.
  - Al intentar crear un segundo elemento que requiera persistencia de metadata en el índice o datos, el sistema verifica que los sectores libres remanentes son insuficientes.
  - La consola integrada bloquea la transacción e imprime en texto de color rojo (FG\_ERROR): *"Error: No hay suficiente espacio en el disco virtual para realizar esta operación."*
- Resultados Obtenidos: Exitoso. Las rutinas numéricas de disco.py (sectores\_libres(), esta\_lleno()) truncan las operaciones a tiempo e impiden que el archivo físico se corrompa por desbordamiento de arreglos.

## 6. MANUAL DE USUARIO

### Requisitos de Entorno e Instalación

Para compilar y ejecutar el simulador de File System, la computadora debe disponer de los siguientes componentes básicos:

1. Sistema Operativo: Multiplataforma (Linux, macOS o Microsoft Windows).
2. Lenguaje de Programación: Python 3.8 o superior instalado globalmente y configurado en las variables de entorno de la terminal (PATH).
3. Librerías Requeridas: No requiere la instalación de gestores de paquetes externos (pip), dado que el núcleo gráfico y estructural depende de módulos nativos estandarizados:
  - tkinter (Librería de renderizado de la GUI).
  - struct (Mapeo de estructuras binarias).
  - json (Serialización de metadatos del árbol).
  - shlex (Tokenización avanzada de strings en comandos para respetar cadenas entrecomilladas).

### Instrucciones de Ejecución

Abre la consola de comandos del sistema operativo, navega hasta el directorio raíz donde se ubican los archivos fuentes del proyecto extraídos y ejecuta la instrucción de inicio:

```
Bash
python main.py
```

### Guía de Operación de la Interfaz Gráfica

La pantalla principal se segmenta de forma fija en dos grandes módulos funcionales distribuidos simétricamente:

1. Panel Lateral Izquierdo (Visualizador TREE): Muestra de forma constante y arborescente el estado lógico completo de todo el File System. Los directorios incorporan el ícono  y los archivos el ícono . El directorio en el cual el usuario se encuentra navegando actualmente se resalta de forma automática en color verde brillante.
2. Panel Central Derecho (Consola Interactiva): Una terminal con fondo negro que simula un entorno CLI de Linux. El usuario tipea las instrucciones en la línea inferior de entrada de texto (prompt) y pulsa Enter para enviarlas.

## Catálogo Completo de Comandos del Sistema

- CREATE <nombre\_disco.fs> <cantidad\_sectores> <tamano\_sector>
  - *Uso:* Inicializa o monta un disco virtual. Si el archivo especificado no existe en la PC, lo crea desde cero con las dimensiones pasadas. Si ya existe, lo abre y carga la sesión previa de forma directa ignorando los otros parámetros numéricos.
  - *Ejemplo:* CREATE midisco.fs 120 512
- MKDIR <nombre\_directorio>
  - *Uso:* Crea una carpeta vacía bajo el nivel de navegación actual.
  - *Ejemplo:* MKDIR fotos\_paseo
- FILE <nombre> <extension> "[contenido\_texto]"
  - *Uso:* Genera un archivo con metadatos específicos y el string de texto provisto. El contenido debe delimitarse preferiblemente entre comillas dobles si posee espacios.
  - *Ejemplo:* FILE notas.txt "Revisar la materia de paginacion de memoria"
- CD <ruta\_destino> | CAMBIARDIR <ruta\_destino>
  - *Uso:* Modifica la posición actual del puntero del terminal. Soporta rutas absolutas (iniciando con /), relativas directas, y saltos de retorno al padre inmediato mediante la sintaxis de doble punto (CD ..).
  - *Ejemplo:* CD /root/fotos\_paseo o CD ..
- LS | LISTARDIR
  - *Uso:* Imprime un reporte detallado ordenado alfabéticamente dentro de la consola con la lista de elementos en la ruta actual, discriminando explícitamente tamaños de bytes.
- MODFILE <nombre.extension> "[nuevo\_contenido]"
  - *Uso:* Modifica y reemplaza de manera íntegra el contenido textual de un archivo preexistente, recalculando de inmediato los bloques de la FAT física.
  - *Ejemplo:* MODFILE notas.txt "Nuevo texto actualizado"
- VERPROPIEDADES <nombre\_elemento>
  - *Uso:* Despliega un desglose estricto con todos los metadatos de un nodo: Tipo, Extensión, Tamaño físico, Fecha exacta de creación y Fecha del último cambio estructural.
- VERFILE <nombre.extension>
  - *Uso:* Abre un flujo de lectura secuencial sobre la FAT y vuelca el contenido del archivo de texto en la consola de la interfaz gráfica.
- COPY <origen> <destino>
  - *Uso:* Comando versátil de clonación de datos. Reconoce tres modalidades operacionales mediante la palabra clave real://:

1. *Real a Virtual*: COPY real://C:/datos.txt informe.txt (Importa un archivo del disco de la PC al File System virtual).
  2. *Virtual a Real*: COPY informe.txt real://C:/usuarios/descargas/informe.txt (Exporta un archivo virtual al almacenamiento del host real).
  3. *Virtual a Virtual*: COPY informe.txt /root/tareas/copia\_informe.txt (Duplica un elemento internamente dentro del árbol virtual).
- MV | MOVER <origen> <destino>
    - *Uso*: Traslada un elemento (archivo o carpeta recursiva) a una nueva ruta de destino. También cumple la función de renombrar un archivo si se ejecuta dentro de la misma carpeta con otro string identificador.
    - *Ejemplo*: MV notas.txt bitacora.txt
  - RM | REMOVE <nombre\_elemento> [-r]
    - *Uso*: Elimina permanentemente los nodos y limpia las entradas correspondientes en la FAT en disco. Para borrar carpetas con elementos adentro, es obligatorio agregar el flag -r (Borrado recursivo).
    - *Ejemplo*: REMOVE notas.txt o REMOVE /root/tareas -r
  - FIND <patrón>
    - *Uso*: Realiza un rastreo recursivo en profundidad desde la raíz localizando coincidencias. Acepta el uso de comodines de tipo asterisco (\*).
    - *Ejemplo*: FIND \*.txt (Retorna las rutas absolutas de todos los archivos con extensión de texto).
  - ESTADO
    - *Uso*: Reporta un resumen analítico completo sobre la salud y capacidad física del disco simulado: Sectores totales, sectores usados, sectores libres, tamaño del sector, bytes disponibles netos y una barra de progreso visual de la tasa de ocupación actual.
  - HELP
    - *Uso*: Despliega una guía rápida en pantalla con la sintaxis exacta de cada operación soportada.
  - CLEAR
    - *Uso*: Limpia los logs informativos acumulados de la terminal gráfica.

## 7. BITÁCORA DE TRABAJO

El desarrollo del proyecto se dosificó a lo largo de un ciclo iterativo de tres semanas, documentando los hitos esenciales a continuación:

- Semana 1 (Diseño de Capas y Serialización Binaria):
  - Análisis detallado de la especificación técnica en PDF dada por la cátedra.
  - Codificación y pruebas iniciales del archivo disco.py. Diseño del empaquetamiento del Header (4sIII) mediante el uso de la librería estándar struct.
  - Definición teórica del valor numérico de las constantes fijas de control (FAT\_LIBRE = -2 y FAT\_EOF = -1).
  - *Verificación / Consulta*: Sesión de aclaración en horas de consulta con la Ing. Erika Marín respecto al almacenamiento jerárquico de carpetas en el binario; se optó por la estrategia óptima de serializar el mapa completo del árbol

estructurado en JSON para persistirlo en el bloque del índice de inicio al desmontar el volumen.

- Semana 2 (Lógica Lógica en Memoria y Despacho de Comandos):
  - Construcción de las clases base de herencia en modelos.py (Nodo, Archivo, Directorio). Implementación del algoritmo recursivo para la reconstrucción dinámica de rutas mediante ruta\_completa().
  - Desarrollo del núcleo del sistema en filesystem.py. Escritura de los algoritmos de asignación encadenada (*Linked Allocation*) y búsqueda lineal por primer ajuste (*First Fit*).
  - Creación del analizador sintáctico en comandos.py implementando una tabla de hash para el despacho de funciones asociadas a strings de comandos (create, file, mkdir, cd, ls, etc.). Integración de shlex.split para procesar de forma nativa parámetros complejos de texto encerrados entre comillas.
- Semana 3 (Capa Gráfica GUI, Casos de Prueba y Optimización):
  - Diseño de la interfaz gráfica con la librería tkinter en interfaz.py. Configuración de la división visual permanente para dar cumplimiento estricto al requerimiento del árbol TREE estático y legible en vivo.
  - Inyección del callback funcional messagebox.askyesno dentro de las rutinas de validación de comandos.py para condicionar de manera interactiva las advertencias de duplicidad de nombres en disco ("caer encima").
  - Depuración de errores en cascada durante la remoción recursiva de elementos (REMOVE -r) para certificar que todos los punteros liberados en memoria se traduzcan en entradas limpias (-2) en la FAT física de bytes del disco.
  - Pruebas de integración cruzadas de importación/exportación (COPY real://) con tipos de archivos del sistema operativo host. Redacción final de la presente especificación técnica.

## 8. BIBLIOGRAFÍA Y FUENTES DIGITALES

1. Silberschatz, A., Galvin, P. B., & Gagne, G. (2018). *Operating System Concepts* (10th ed.). Wiley. (Capítulos específicos sobre administración de almacenamiento masivo, Sistemas de Archivos y arquitecturas de asignación basadas en FAT y listas enlazadas).
2. Python Software Foundation. (2026). *The Python Standard Library Documentation*. Recuperado de <https://docs.python.org/3/> (Documentación oficial sobre los módulos de persistencia binaria struct, procesamiento de cadenas shlex y diseño gráfico nativo mediante tkinter).
3. Tanenbaum, A. S., & Bos, H. (2015). *Modern Operating Systems* (4th ed.). Pearson. (Análisis comparativo de algoritmos de ubicación de espacio en bloques de memoria y fragmentación interna/externa).